

NeurIPS 2022 | Train on Your Own Mistakes: A Simple Fix for the Open-Loop/Closed-Loop Gap in Self-Driving

Woven Planet's CW-ERM runs a policy once in a simulator, finds the scenes where it fails, upsamples them, and retrain, cutting collisions by up to ~35% with no differentiable simulator.

Most self-driving planners are trained by behavioral cloning: show the network what a human expert did, and ask it to reproduce the expert's next action. The training is done open-loop, one step at a time, matching expert actions on logged data. But when the policy actually drives, it runs closed-loop: every action it takes changes the state it sees next. A small steering error nudges the car into a slightly off-center position the expert never visited, the next prediction is a little worse, and the errors compound. This is covariate shift, and it is why a planner that looks excellent on open-loop metrics can still drift into collisions on the road.

There is a second, quieter problem. The metrics people actually care about (did the car collide, did it leave its lane) are non-differentiable. The training loss cannot see them directly, so open-loop training and closed-loop deployment end up optimizing two different objectives. The obvious remedy is to put a simulator inside the training loop and backpropagate through it, as methods like Urban Driver do. It works, but it is expensive: it needs a differentiable simulator, it does not scale well, and unrolling the policy during training is memory-hungry.

This NeurIPS 2022 paper from Woven Planet asks a simpler question. What if the simulator is used only to decide which training scenes matter, and never touches the loss function? Their answer is CW-ERM, or Closed-loop Weighted Empirical Risk Minimization: a two-step recipe that finds the scenes a policy fails on and upsamples exactly those scenes when training the final policy. The whole method is captured in the figure below.

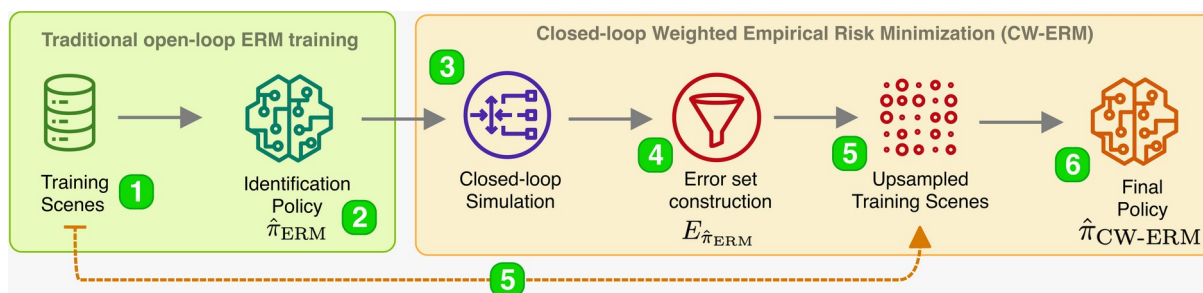


Figure 1. High-level overview of CW-ERM. Steps 1-2: train an identification policy with ordinary ERM. Step 3-4: roll it out in a closed-loop simulator and collect a cost metric (e.g. collisions) to build the error set. Steps 5-6: upsample the error-set scenes and retrain the final policy on the reweighted data.

Paper: <https://arxiv.org/abs/2210.02174> · Code: <https://github.com/wp-research-uk/cw-erm>

Why open-loop training quietly hurts closed-loop driving

Empirical Risk Minimization (ERM) is the standard behavioral-cloning objective: minimize the expected per-step imitation loss over states drawn from the expert's own distribution. The policy is only ever asked about states the expert visited, so it never learns to recover from its own mistakes. At deployment the distribution flips, since the states are now drawn from the policy's own rollouts, and the mismatch between these two distributions is precisely what makes the car unsafe. Because collisions are non-differentiable, the ERM loss offers no gradient that pushes the policy toward avoiding them. The authors' insight is that you do not need a gradient through the metric; you only need to know which scenes the policy gets wrong, and you can get that from a plain, non-differentiable simulator.

The method: three stages, one weighting term

CW-ERM is deliberately simple and breaks into three stages. First, train an identification policy the ordinary way, with standard ERM behavioral cloning. Second, roll that policy out in a closed-loop simulator across the training scenes and record a cost such as the number of collisions; every scene where the cost is positive, meaning every scene the policy fails, goes into an error set. Third, train the final policy again with weighted ERM, where the error-set scenes are upsampled by a factor w . The final objective is almost identical to plain behavioral cloning, with the single addition of a weight that is driven entirely by closed-loop failures.

Two practical details make it work. The identification policy is stopped early, after K epochs, so that it still fails often enough to populate a meaningful error set; train it to convergence and the error set is depleted. And rather than reweighting the loss, the authors physically upsample the failing scenes by an integer factor, which they find more stable. Because the simulator is used only to label scenes and never to compute a gradient, any non-differentiable metric can drive the weighting, and no differentiable simulator is needed. The paper also connects this reweight-by-failure scheme to the classic idea of correcting covariate shift through density-ratio estimation, giving the recipe a principled backbone.

Results: fewer collisions across the board

The method is evaluated on a proprietary real-world urban driving dataset collected by Woven Planet's self-driving cars on challenging missions in San Francisco and Palo Alto, complete with recorded ego and agent trajectories and HD maps. The scenes are hard and varied (stopping behind a lead vehicle, holding at intersections, threading dense traffic with pedestrians and cyclists) and run 11 to 13 seconds each, up to 30 seconds at the longest. Training uses 180 hours of driving, with 60 hours each held out for validation and test, and the final policy is trained for 40 epochs. The closed-loop simulator and metrics are released as open source.

Compared against the strongest baseline, behavioral cloning with ERM and expert perturbation, CW-ERM reduces collisions significantly across every metric, with improvements reaching about 35% on some of them. When the model upsamples front-collision scenes, front collisions fall from 14 to 9, roughly a 36% reduction. When it upsamples side-collision scenes, side collisions fall from 55 to 47. There is a welcome side effect: improving one metric often

improves related ones. Upsampling side collisions also reduces rear collisions, a sign that the policy is becoming less passively cautious rather than gaming a single number. Variance is also lower than the baseline in several cases, so the gains are stable rather than lucky. Figure 2 shows the full picture across metrics and CW-ERM variants.

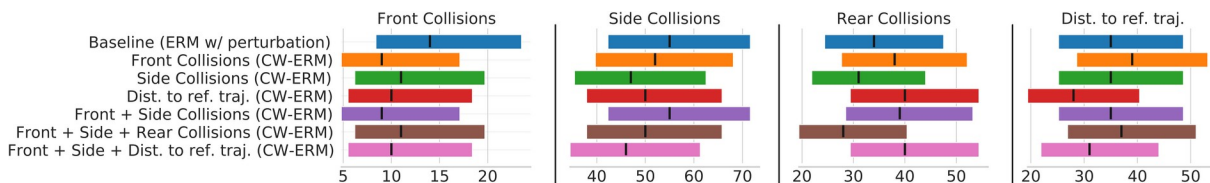


Figure 2. Closed-loop evaluation across four collision metrics (front, side, rear, distance-to-reference). Each row is a CW-ERM variant that upsamples a different metric; the top row is the best baseline (ERM with perturbation). Bars show 0.95 confidence intervals from an exact Binomial posterior. Targeting a metric shrinks it, and improvements often carry over to related metrics.

The qualitative behavior is just as telling as the aggregate numbers. Figure 3 shows a test scene where the baseline drives into an intersection, ignores a car approaching from the right, and causes a front collision; the CW-ERM policy, trained with front-collision scenes upsampled, instead waits at the intersection until the path is clear. The change is not a cosmetic trajectory tweak: it is the policy learning to yield in exactly the situation type it used to fail on.

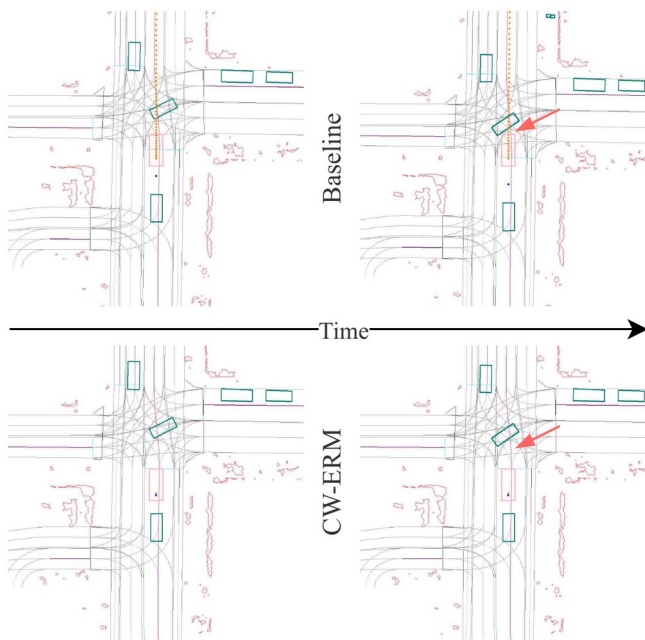


Figure 3. A test scene, baseline (top) versus CW-ERM with front collisions upsampled (bottom), left to right in time. Blue dots are the target trajectory, yellow dots the prediction; the ego is the pink box, other agents the teal boxes. The baseline enters the intersection and collides with the car from the right (red arrow); CW-ERM waits instead.

How hard to upsample, and which metrics to combine

Two knobs govern how well the recipe works. The first is the upsampling factor w . Figure 4 sweeps it against side collisions on the validation set: performance improves steadily up to about $w = 50$, after which side collisions start to climb again, the same saturation the Just Train

Twice (JTT) method reports for its upsampling scheme. The second knob is the early-stopping budget K of the identification policy: about 10 epochs works best for single-metric error sets and 20 for multi-metric ones. Together these give a small, legible search space rather than a fragile one.

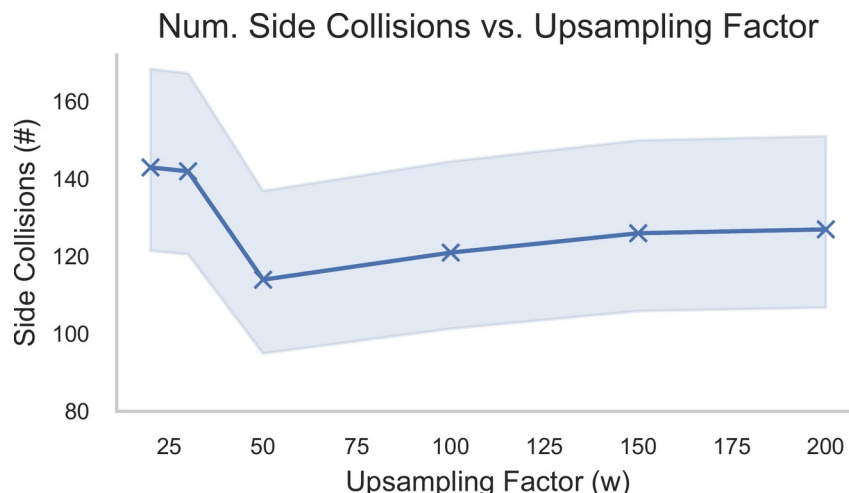


Figure 4. Side collisions on the validation set as a function of the upsampling factor w . Performance improves up to about $w = 50$ and degrades beyond it; the shaded band is the confidence interval.

There is also a choice of which failures to target. Upsampling a single metric gives the largest gain on that metric, while combining metrics trades them off along a Pareto front, so the right choice depends on which failure a given deployment cares about most. Combining front, side, and distance-to-reference works nicely, but adding rear collisions causes a clear regression. The authors trace this to false positives in the rear-collision metric: the simulator replays other agents from logs, so a log-replayed car that would have braked in reality drives straight into the ego and is scored as the ego's fault. The practical lesson is that the method is only as trustworthy as the metric it upsamples, and a noisy metric can actively steer training in the wrong direction. Distance-to-reference failures, meanwhile, fall from 35 to 28 against the best baseline, about a 20% reduction, and the reweighting leaves inference untouched, so none of this costs anything at drive time. Figure 5 shows a matching qualitative save on the side-collision axis: the baseline drifts off its target trajectory and hits a bus ahead, while the CW-ERM policy slows down behind the bus and avoids the collision.

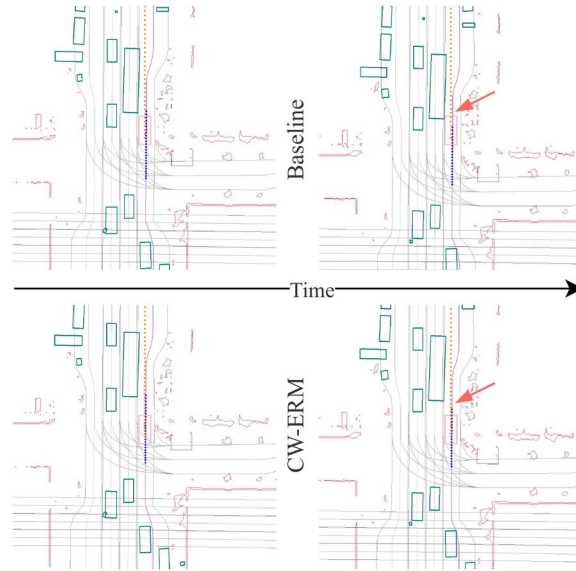


Figure 5. Baseline (top) versus CW-ERM with side collisions upsampled (bottom), over time. The baseline deviates from the target trajectory and collides with the bus ahead (red arrow); CW-ERM slows down behind the bus and prevents the collision.

A compact scoreboard

The headline comparisons against the strongest baseline are easy to summarize. Each CW-ERM number comes from the variant that upsamples the matching metric.

Table 1. CW-ERM versus the best baseline (ERM with perturbation) on the test set. Lower is better.

Metric	Baseline	CW-ERM	Change
Front collisions	14	9	~36% fewer
Side collisions	55	47	fewer
Distance-to-reference failures	35	28	~20% fewer

Why it matters

The appeal of CW-ERM is how little it asks for. You do not need a differentiable simulator, a human in the loop, or expensive closed-loop training to capture closed-loop benefits. You run your policy once in a simulator, note where it fails, upsample those scenes, and retrain. The recipe delivers meaningful collision reductions, works with any closed-loop metric, and adds no inference latency, since all the extra work happens at training time. The honest boundaries are worth stating too: the gains are measured on one company's proprietary urban dataset, the rear-collision result exposes how sensitive the method is to metric quality, and the simulator's log-replayed agents can manufacture failures that are not really the ego's fault. Even so, as a cheap, metric-agnostic, and theoretically grounded way to nudge an imitation-learned planner toward safer closed-loop behavior, CW-ERM is a genuinely practical idea that a team could adopt without much friction.